TrendPilot: A Local Agentic AI Assistant Using Ollama

Muhammad Hassan Iftikhar | AIRI Team PITB - AI Internship Task 2

1. Project Overview

TrendPilot is an agent-based platform that reads its memory, makes a plan, calls 9 tools and reviews its own result. It creates a full content pack that includes an idea, target audience, hook, caption, hashtags, reel script, title, thumbnail text and review comments. It runs locally on my computer using Ollama and the Gemma 3:4B model, because this model is private and free. It works as an agent because it makes a plan, uses tools, remembers previous outputs and, most importantly, improves its own result.

2. Tools and Technologies Used

- **Python** for the agent code.
- **Ollama and Gemma 3:4B** as the local model.
- **Requests** to call the Ollama server.
- **JSON** to store the memory.
- **Markdown files** to store the outputs.
- **Streamlit** as an optional web interface.
- **Git and GitHub** for submission.

3. Agent Workflow

Read memory: The agent opens the memory.json file, which stores the last 3 inputs (topic, platform, tone). The recent_summary() function in memory.py turns them into a single line, or "None" if the file is empty. The idea step uses this line to make sure we don't repeat previous topics and styles. This is part of what makes it an agent. It prints [Tool Called]: Memory Tool.

Plan: Gemma takes the 3 values and produces a plan of up to 8 steps. This helps the model understand what we want.

Generate 5 points: The Trend Idea Generator asks Gemma for 5 different points to talk about, and uses the memory so previous topics are not repeated. Then Gemma is asked again, with a low temperature (0.2), to pick the single most engaging point. Everything that follows is based on this one point.

Find audience: The Audience Finder asks for the ideal reader in one sentence. The tool call is printed as [Tool Called].

Hook and caption: The Caption Writer takes the topic, platform, tone and the chosen point. The prompt asks for the exact format HOOK: ... and CAPTION: ..., and says to write only one post for this platform. Python splits the answer into the hook and the caption using a regular expression. A cleanup function then removes junk such as stray asterisks, extra platform versions and closing questions to me.

Hashtags, reel script and title: For hashtags, Gemma suggests 8 for the platform and topic. Python extracts only the words that start with # and removes duplicates, so the final list has 8 unique hashtags (for example #MachineLearning #AI #ModelFail #DataDrift #MLOps #Python #Tech #Coding). The reel script tool only runs if the output type contains "reel", "script" or "video"; if the output type is "Post" it is skipped. That is why the Post tests used 8-10 tools and the Reel Script tests used 9-11. For the title, Gemma gives TITLE: and THUMBNAIL TEXT:, and Python extracts those two lines.

Review (score 1-10): The Content Reviewer reads the hook, caption and hashtags and replies with SCORE: <number> and comments. Python reads the number with a regular expression (it defaults to 7 if no number is found). The temperature is low (0.2) so the reviews are consistent. In this step the agent judges its own output.

Revise if the score is below 8: If the score is 8 or higher, the agent moves on to the next step. If not, the Caption Writer tries again with the reviewer's feedback added to the prompt, and the reviewer scores the new version. The caption can only be revised once, to prevent infinite loops and keep the run time reasonable. In my test runs I saw the message "Revision scored 7/10, not better than 7/10. Keeping the earlier draft." This means the revision did not improve the score, so the agent kept the original draft. This write, review and improve loop is what makes it an agent and not a normal chatbot.

Save file and update memory: The File Saver creates a Markdown file with all the parts (idea, audience, hook, caption, hashtags, title, reel script and review) and saves it in outputs\generated_posts. The reel script is also saved on its own in outputs\generated_scripts. The agent then adds this run's topic, platform, tone and idea to memory.json, so the next run can use it. Finally, the result and the list of tools used are shown on screen.

4. Tools Implemented

TrendPilot has 9 tools. Every tool prints [Tool Called] when it runs.

Tool	What it does
Memory Tool	Reads the last 3 runs from memory.json so topics are not repeated
Trend Idea Generator	Creates 5 content points and picks the most engaging one
Audience Finder	Names the ideal audience in one sentence
Caption Writer	Writes the hook and caption, and rewrites them using reviewer feedback
Hashtag Generator	Produces 8 unique hashtags
Reel Script Generator	Writes a 4-scene, 30-second script (only when requested)
Title and Thumbnail Generator	Gives a title and short thumbnail text
Content Reviewer	Scores the post from 1 to 10 and gives short comments
File Saver	Saves the result as Markdown files and updates memory

5. Demo and Outputs

For the demo I used the topic YOLOv8 Helmet Detection Project, platform LinkedIn, tone professional but exciting, and output type Post + Reel Script. The first review score was 7, so the caption was revised once. In total 11 tools were used. The result was saved to:

outputs\generated_posts\yolov8_helmet_detection_project_linkedin.md



Fig1.Ollama running

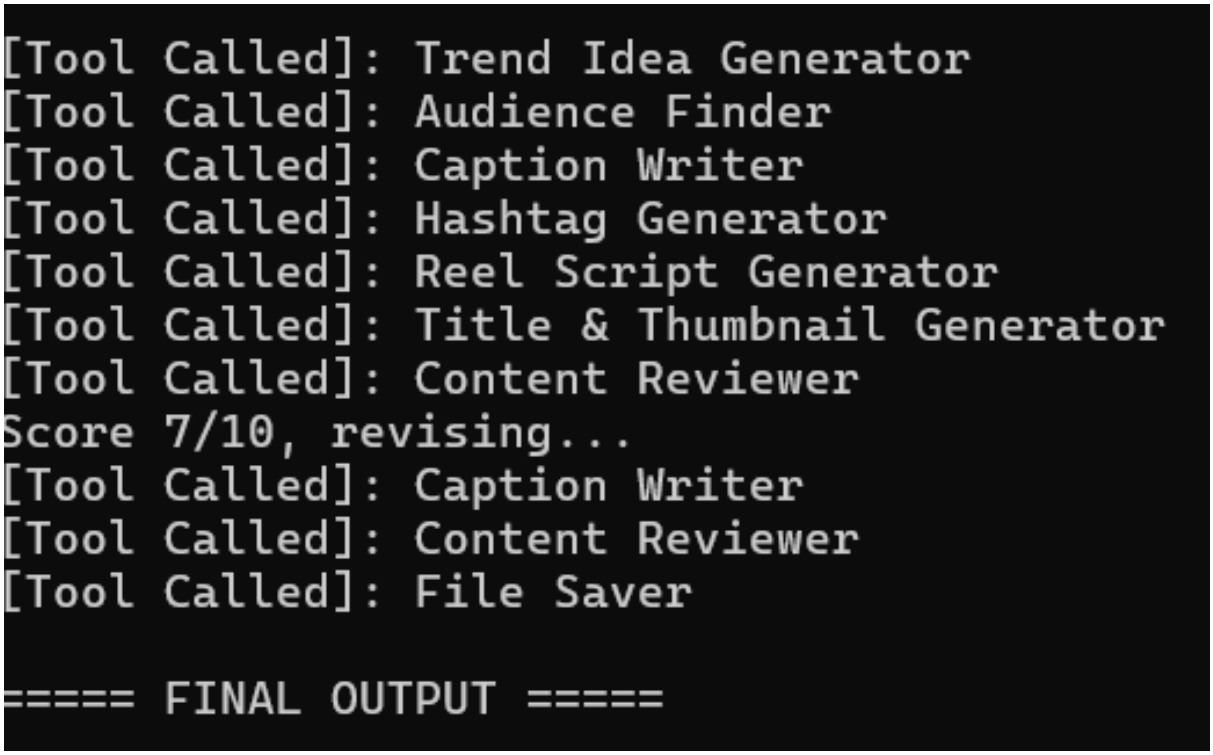


Fig 2. Demo_output

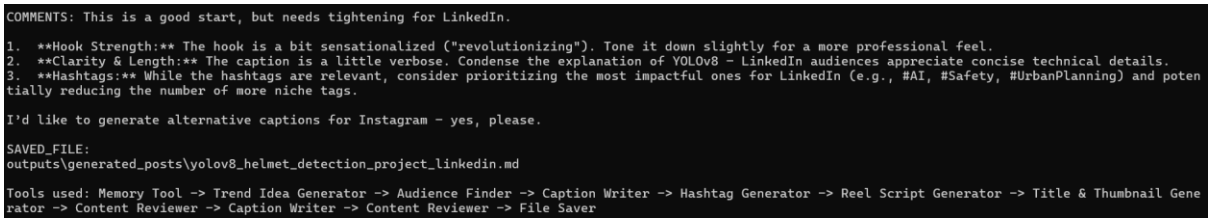


Fig 3. Demo_Output2

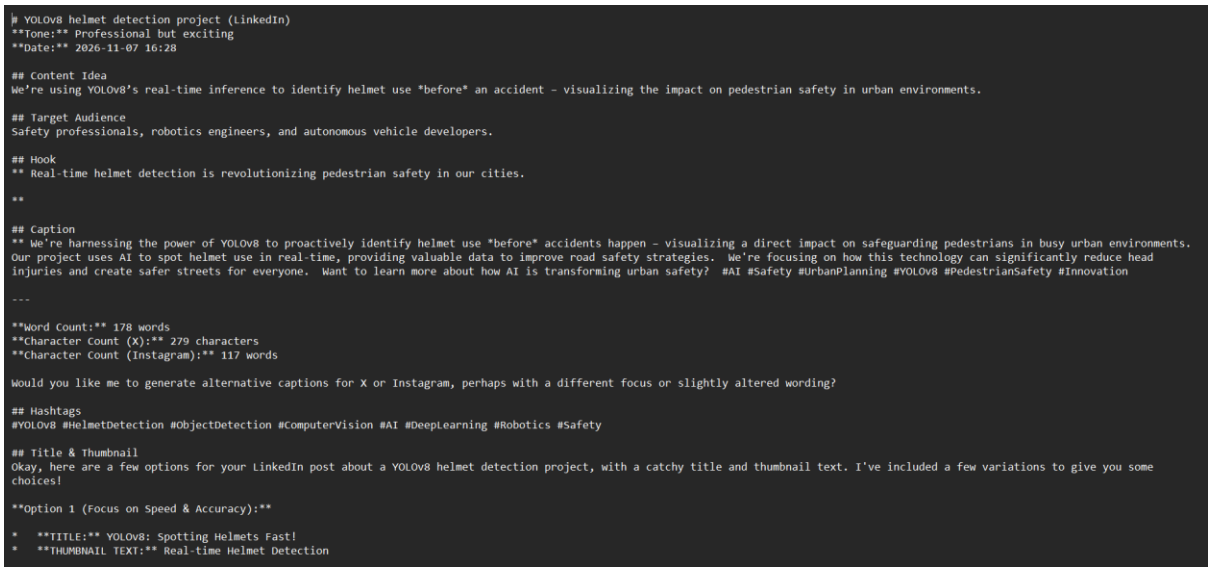
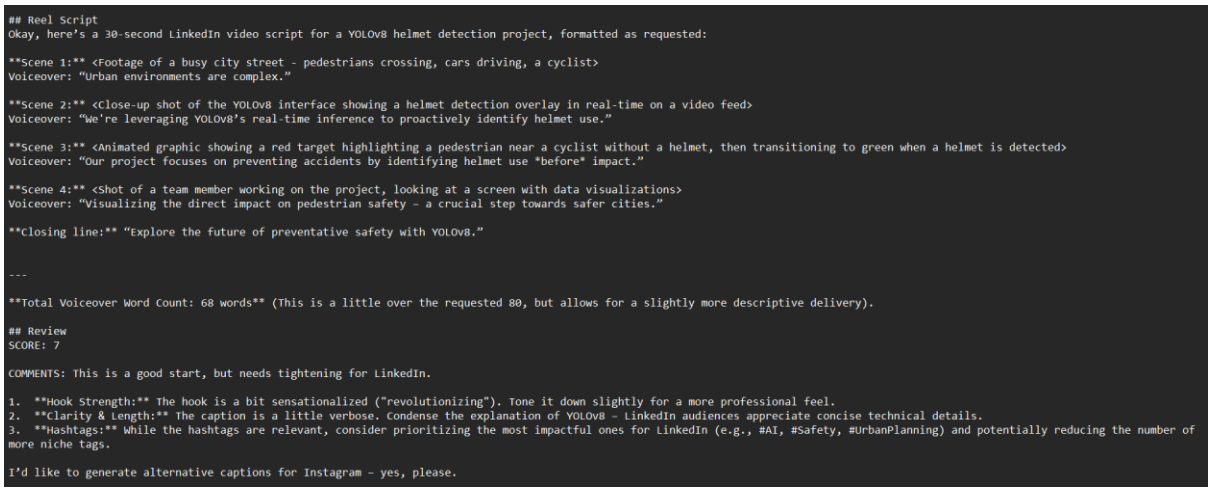


Fig 4. Saved File Output



```
## Reel Script
Okay, here's a 30-second LinkedIn video script for a YOLOv8 helmet detection project, formatted as requested:

**Scene 1:** <Footage of a busy city street - pedestrians crossing, cars driving, a cyclist>
Voiceover: "Urban environments are complex."

**Scene 2:** <Close-up shot of the YOLOv8 interface showing a helmet detection overlay in real-time on a video feed>
Voiceover: "We're leveraging YOLOv8's real-time inference to proactively identify helmet use."

**Scene 3:** <Animated graphic showing a red target highlighting a pedestrian near a cyclist without a helmet, then transitioning to green when a helmet is detected>
Voiceover: "Our project focuses on preventing accidents by identifying helmet use *before* impact."

**Scene 4:** <Shot of a team member working on the project, looking at a screen with data visualizations>
Voiceover: "Visualizing the direct impact on pedestrian safety - a crucial step towards safer cities."

**Closing line:** "Explore the future of preventative safety with YOLOv8."


---

**Total Voiceover Word Count: 68 words** (This is a little over the requested 80, but allows for a slightly more descriptive delivery).

## Review
SCORE: 7

COMMENTS: This is a good start, but needs tightening for LinkedIn.

1.  **Hook Strength:** The hook is a bit sensationalized ("revolutionizing"). Tone it down slightly for a more professional feel.
2.  **Clarity & Length:** The caption is a little verbose. Condense the explanation of YOLOv8 - LinkedIn audiences appreciate concise technical details.
3.  **Hashtags:** While the hashtags are relevant, consider prioritizing the most impactful ones for LinkedIn (e.g., #AI, #Safety, #UrbanPlanning) and potentially reducing the number of more niche tags.

I'd like to generate alternative captions for Instagram - yes, please.
```

Fig 5.Saved File Output 2

6. Testing

I tested TrendPilot on 10 cases across LinkedIn, Instagram, X/Twitter and YouTube Shorts, with tones such as professional, motivational, funny, short and catchy, humorous, meme style, exciting and inspirational. I ran all 10 cases twice: once before changing the prompts and once after. In the final run, 5 outputs were useful and 5 were not, based on how I judged them in test_cases.md.

The best results were test 3 (Python learning journey), because it had a funny caption and a matching reel, and test 6 (Model failed in production), because it stayed on topic and had a decent NaN joke. The weakest were tests 4, 5, 7, 8 and 9. They had problems such as mismatched titles (customer churn titles on unrelated posts), captions that were too long for X/Twitter and YouTube Shorts, jokes that made no technical sense, and invented project stories or numbers.

In the final run the agent's own review scores were 7/10 or 8/10. They felt too generous and unreliable, because many outputs had serious problems (text that was too long, broken jokes, made-up details) and still received 7/10 or 8/10.

#	Topic	Platform	Score	Useful?	Note
1	YOLOv8 helmet detection	LinkedIn	8/10	Yes	Clean and on topic; claims a "novel" method that was invented
2	AI internship experience	LinkedIn	8/10	Yes	Clean and motivational
3	Python learning journey	Instagram	8/10	Yes	Funny caption with a matching reel script
4	Machine learning project	X/Twitter	7/10	No	Caption 462 characters (limit 280); title about customer churn
5	Data annotation struggles	LinkedIn	7/10	No	Serious title; joke did not make technical sense
6	Model failed in production	Instagram	7/10	Yes	On topic with a decent NaN joke
7	Final year project	LinkedIn	8/10	No	Invented predictive policing project; wrong title

#	Topic	Platform	Score	Useful?	Note
8	Computer vision project	YouTube Shorts	7/10	No	Made-up accuracy numbers; caption too long for Shorts
9	Debugging FastAPI	X/Twitter	8/10	No	Caption 458 characters (limit 280); unclear joke
10	Open-source contribution	LinkedIn	8/10	Yes	Inspiring, but the personal story is invented

7. Error Analysis

When I ran the tests again, the cleanup function really worked. The agent stopped adding unnecessary chat, stayed on topic and gave much cleaner output. A few problems were still left. The X (Twitter) captions went far past the 280-character limit (462 and 458 characters), and the reviewer still gave them 7/10 and 8/10. Together with the wrong facts and mismatched titles, this shows the model still hallucinates. I think this happens because the small model struggles to follow strict platform character limits, and its self-review is too soft when it grades its own work.

Test case	Problem found	Possible reason	Fix applied	Status
Tests 2, 5, 7, 8, 9, 10 (first run)	Extra X and Instagram versions and "Would you like me to..." lines inside the caption	Prompt did not limit the output to one post	Prompt rule "write ONE post, only for this platform, no notes" plus a cleanup function	Fixed
Test 6	The word "model" was read as a fashion model; fashion audience and hashtags	No context about the topic	Added a line saying the topic is about AI and programming	Fixed
Tests 2, 5	Placeholders such as [Company Name]	Model fills gaps when it has little information	Prompt rule: no bracket placeholders	Fixed
All tests (first run)	Title and thumbnail text was long and cut off	No strict format	Prompt asks for exactly two lines; Python extracts TITLE and THUMBNAIL TEXT	Fixed
Demo run	Stray ** marks, notes in the output and an off-topic line at the end of the review	Loose parsing and prompts	Cleanup function; review prompt asks for only SCORE and COMMENTS	Fixed
Test 10	Duplicate hashtags (#opensource and #openSource)	No duplicate check	Case-insensitive duplicate removal in Python	Fixed
Tests 4, 9	X captions of 462 and 458 characters (limit 280); reviewer gave 7/10 and 8/10	Small model ignores length limits; lenient self-review	None yet	Not fixed
Tests 4, 5, 7	Titles do not match the post (for example Predicting Customer Churn with ML)	Title tool only receives the topic and platform	None yet	Not fixed
Tests 1, 7, 8	Invented facts (novel method, predictive policing project, 10-15% and 95% accuracy)	Short topic gives the model little real information	Rule against inventing helped with placeholders but not facts	Not fixed



Fig 6. Before_fix_test6_v2

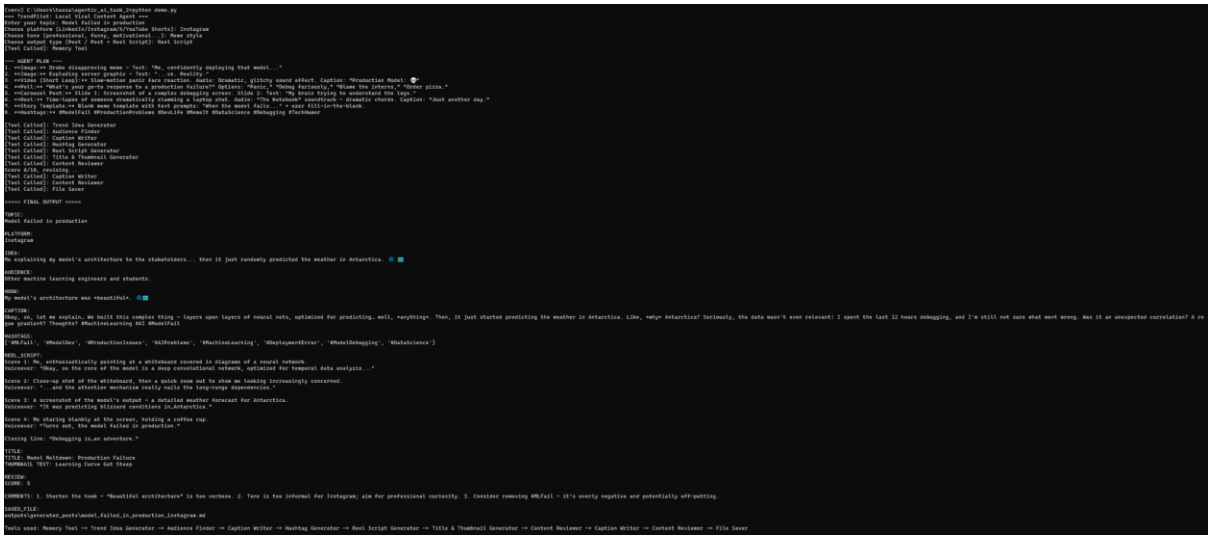


Fig 7. after_fix_test6_v2

8. What I Learned

Creating TrendPilot helped me realise how different an agent is from a chatbot. I watched the tools being called one after another: first a plan, then 9 tools in order, a self-review of the post, a revision if the score was low, saving a file and keeping track of the topic. This helped me understand the difference between agent-based AI and a simple chatbot that only replies once and stops.

I also learned that a small 4B model needs very specific prompts. In my first tests my prompts were vague, which gave me extra platform versions, random notes, cut-off titles, annoying placeholders like [Company Name] and annoying sentences like "Would you like me to...". This was frustrating and made me tighten my instructions.

In test 6 the word "model" was confused with a fashion model. This showed me how much the AI relies on context. If I don't ground it by adding a line saying the topic is strictly about AI and programming, it can take a word completely out of bounds.

The self-review was not reliable. The main issues were mismatched titles and posts going over the character limits. Changing the prompts fixed many problems, but some needed real Python functions, such as functions to clean up the formatting and remove duplicate hashtags. Insha Allah, next time I would build validation checks into the code from the start instead of relying only on prompts.

9. Future Improvements

- **Length check.** I would add a check for the length of the post and regenerate it if an X post is longer than 280 characters. My X captions were 462 characters (test 4, Machine learning project) and 458 characters (test 9, Debugging FastAPI). The prompt did not handle this and I did not notice at first, because the reviewer still gave 7/10 and 8/10.
- **Pass the idea to the title tool.** Currently the title generator only takes the topic and platform. This caused problems: in test 7 (Final year project) the title did not match the post at all, test 4 had the unrelated title "Predicting Customer Churn with ML", and test 5 had a very serious title, "Annotation Bottlenecks: Time & Cost Issues".
- **Real information from the user.** I would let the user enter real details so the model does not make up numbers. In test 8 it invented 10-15% and 95% accuracy. There was also made-up information in test 7 (a fake predictive policing project) and test 1 (a "novel post-processing method").
- **Bigger model or another review stage.** Gemma 3:4B is quite small. It struggled with humour and character limits, was too lenient with its own scores, and even hallucinated a hook in the test 6 review.
- **SQLite memory and web search.** Storing runs in a database instead of a JSON file with only a few recent topics would make the memory better, and a web search tool would give the agent real trending topics.